Module 4: Enhancing Web Presentation

Introduction to HTML

Module Objectives

Module Title: Introduction to HTML

Module Objective: This module focuses on improving the presentation of your web pages using advanced HTML styling techniques, global attributes, and event handling. You'll explore layout concepts and learn how to make your pages more interactive and user-friendly.

Skills you'll gain: Styling and layout fundamentals, using global and event attributes, and enhancing user interaction.

At the end of this module, you'll complete a project that showcases your understanding of HTML styling and layout, followed by a test to wrap up the module.

Module Objectives

Course Syllabus:

The course is organized into six modules, each focusing on key aspects of HTML, along with a final project and test to solidify your learning. Below is a detailed breakdown of the syllabus:

Module 4: Enhancing Web Presentation

- HTML Styling and Layout – Part 1
- HTML Styling and Layout – Part 2
- HTML Styling and Layout – Part 3
- Global Attributes
- Event and Data Attributes
- Module 4 Completion

4.1 HTML Styling and Layout Part 1

4.1.1 How to Apply Styles to HTML Elements

In this part of our course, we're going to learn how to make our HTML pages look nice. We'll look at two simple ways to add style directly to our HTML:

using the style attribute right in the HTML tags (inline styling), and
putting styles inside the HTML document with a style section (internal stylesheet and the <style> element).
While the best practice for styling web pages in most cases involves using external stylesheets and CSS for greater flexibility and maintainability, in this section, we're focusing on adding styles within an HTML document.

This approach not only helps build a strong foundation in styling basics but also prepares you for the CSS techniques we'll discuss in the second course of the Web Dev Essentials series, "CSS Essentials."

4.2 HTML Styling and Layout Part 2

4.2.1 Colors and Font Properties

With the power of HTML (and CSS rules), you can bring your web pages to life by defining colors and font properties to create more visually appealing and user-friendly experiences.

Colors play a vital role in web design, as they evoke emotions and establish visual hierarchy. Font properties, on the other hand, allow you to define the typeface, size, weight, and other characteristics of the text on your web pages.

By combining the two, you'll learn how to enhance the overal aesthetic, readability, and emphasis of your websites.

4.2.2 Colors and Color Formats in HTML

Colors can be applied to HTML elements using the CSS color property or the background-color property. In HTML, colors can be specified in several ways. Here are some of them:

- **by name** – there are 147 color names that can be used, such as "red", "green", "blue", "yellow", "purple", and "black".
- **by hexadecimal value** – a six-digit code preceded by a pound sign (#) that represents a combination of red, green, and blue values. For example, #FF0000 represents the color red.
- **by RGB value** – a set of three values representing the amount of red, green, and blue, respectively. Each value ranges from 0 to 255. For example, RGB(255, 0, 0) represents the color red.

4.2.2 Colors and Color Formats in HTML

For example:

```
<!DOCTYPE html>
<html>
    <head>
        <style>
            h1 {
                color: #FFFFFF;
                background-color: green;
            }

            p {
                color: RGB(0, 0, 255);
            }
        </style>
    </head>
    <body>
        <h1>This is a heading</h1>
        <p>This is a paragraph.</p>
    </body>
</html>
```

In the code, we've defined the colors using the following CSS rules within the <style> element:

For the <h1> element:

- The text color is set to white (the hexadecimal value of #FFFFFF)
- The background color is set to green (the name green)

For the <p> element:

- The text color is set to blue (the RGB value of 0, 0, 255)

When the code is rendered in a web browser, the heading <h1> element will have white text color and a green background color. The paragraph <p> element will have blue text color.

4.2.3 Font Properties

Font properties are an essential aspect of web design, as they help to define the appearance and readability of your text. HTML provides several font properties, including font-size, font-family, font-weight, and font-style:

- The font-family property specifies the font to be used for the text. It can be set to a specific font name, such as Arial, Times New Roman, or Helvetica, or a generic font family such as serif, sans-serif, or monospace. The default value depends on the browser.
- The font-size property defines the size of the font. It can be set using an absolute value such as pixels (px) or points, or a relative value such as percentages (%)or em units (ems). The default value is normal.
- The font-weight property sets the weight, or thickness (boldness), of the font. It can be set to a specific value name such as bold, bolder, or lighter, or to a numeric value between 100 and 900, with 400 being the default value (normal).
- The font-style property defines the style of the font, such as normal (default value), italic, or oblique.

4.3 HTML Styling and Layout Part 3

4.3.1 The <span> and <div> Elements

The <span> and <div> elements are HTML tags that are commonly used for layout and styling purposes in web development. They do not have any inherent meaning or semantic value, but instead serve as containers for other elements.

4.3.2 The <span> Element

The <span> element is an inline element that can be used to apply styles to specific parts of a text or a small group of inline elements. It is often used in conjunction with CSS to style text or small portions of an HTML document.

For example, you might use the <span> element to style a single word in a sentence with a different color, font size or font style:

index.html

```html
<p>This is a <span style="color: green;">sample text</span> with some inline styling.</p>
```

Code/Preview

blank-html-page.html

```html
<p>This is a <span style="color: green;">sample text</span> with some
   inline styling.</p>
```

This is a sample text with some inline styling.

In this example, the `<span>` element is used to style the phrase "sample text" with the color green.

4.3.3 The <div> Element

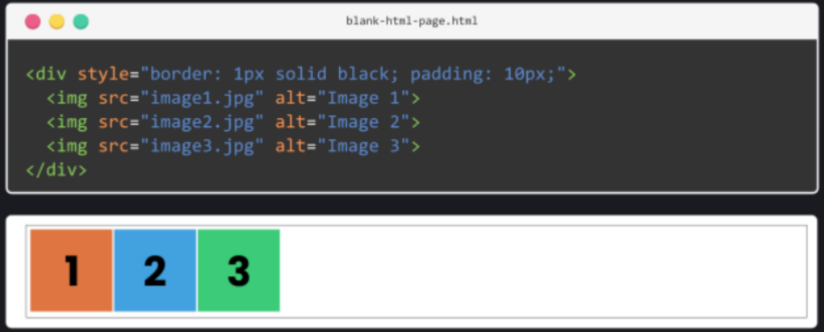
The <div> element is a block-level element that is used to create a container for other HTML elements. It is commonly used to group together a set of elements and apply styles to them as a whole.

For example, you might use a <div> element to group together a set of images and apply a border or padding to the entire group:

index.html

```html
<div style="border: 1px solid black; padding: 10px;">
  <img src="/uploads/media/default/0001/04/rect1.png" alt="Image 1">
  <img src="/uploads/media/default/0001/04/rect2.png" alt="Image 2">
  <img src="/uploads/media/default/0001/04/rect3.png" alt="Image 3">
</div>
```

Code/Preview



4.3.4 When to Use <span> and <div>

While the <span> and <div> elements are non-semantic and should not be used to convey any meaning on their own, they can be useful in certain situations:

1. Use <span> when you want to apply styles to a specific portion of text or a small group of inline elements. For example:

index.html

```html
<p>This is a <span style="color: blue;">blue</span> text.</p>
```

2. Use <div> when you want to group together a set of elements and apply styles to them as a whole. For example:

index.html

```html
<div>
<button>Button 1</button>
<button>Button 2</button>
</div>
```

4.4 Global Attributes

4.4.1 Global Attributes

The <span> and <div> elements are HTML tags that are commonly used for layout and styling purposes in web development. They do not have any inherent meaning or semantic value, but instead serve as containers for other elements.

4.4.1 Global Attributes

Before going further, let's revisit the concept of global attributes in HTML. Global attributes are unique in that they can be added to any HTML element, providing various functionalities throughout your website.

We've already talked about the style attribute, but there are several more that are widely used, such as: id, class, title, accesskey, lang, hidden, dir, translate, and onclick.

4.4.2 The id Attribute

The id attribute assigns a unique identifier to an element, which then can be used for several purposes, including:

- unique identification, which lets you distinguish HTML elements from each other on the page.
- styling, which allows you to apply specific CSS styles to a single element. (By targeting the element's unique id, you can create custom styles that only affect that particular element. You're going to learn about it in the second part of the course.)
- JavaScript manipulation, which lets you select and manipulate specific elements in the DOM. (It makes it easy to access, modify, or perform actions on a particular element without affecting others. Again, you're going to learn more about it in the Web Development Essentials: JavaScript course.)
- anchors, which you can use to create internal (anchor) links within a webpage. (You learned about them in the Links section of this course.)

4.4.2 The id Attribute

Code/Preview

blank-html-page.html

```html
<!DOCTYPE html>
<html>
  <head>
    <style>
      #greeting {
        font-size: 24px;
        color: blue;
      }
    </style>
  </head>
  <body>
    <p>Hello, World!</p>
    <p id="greeting">Hello, world!</p>
    <p>Hello, World!</p>
  </body>
</html>
```

Hello, World!

Hello, world!

Hello, World!

4.4.3 The class Attribute

The class attribute assigns a class to an element, allowing you to group and target multiple elements for various purposes, such as:

grouping elements – the class attribute allows you to group multiple HTML elements that share similar characteristics or purposes. Unlike the id attribute, class can be assigned to multiple elements on a page.

styling – by using the class attribute, you can apply consistent CSS styles to a group of elements. This helps maintain a uniform appearance and promotes reusability of styles across your webpage or even an entire website.

JavaScript manipulation – when working with JavaScript, the class attribute can be used to select and manipulate groups of elements with the same class. You can perform actions or apply changes to all elements with the specified class at once, making it an efficient way to manage similar elements.

readability and maintainability – using class attributes can help improve the readability and maintainability of your HTML and CSS code. By assigning meaningful class names, you can easily understand the purpose or functionality of specific elements, making it easier to update or modify your code in the future.

4.4.3 The class Attribute

Code/Preview

blank-html-page.html

```
<!DOCTYPE html>
<html>
<head>
    <style>
      .highlight {
        background-color: blue;
        color: white;
        font-weight: bold;
      }
    </style>
</head>
<body>
    <p>Here is a normal paragraph.</p>
    <p class="highlight">This paragraph has the 'highlight' class
      applied to it.</p>
    <p>Another normal paragraph.</p>
    <p class="highlight">Another paragraph with the 'highlight' class.</p>
</body>
</html>
```

Here is a normal paragraph.

This paragraph has the 'highlight' class applied to it.

Another normal paragraph.

Another paragraph with the 'highlight' class.

4.4.3 The class Attribute

As you can see, the class attribute acts like a label that you can attach to elements, allowing you to target and style them using CSS. Let's see what happens in the code:

1. We start by defining a CSS rule (a class) in the <style> block. The rule is named .highlight, and it includes properties like background-color, color, and font-weight that determine how elements with this class will look.
2. As we move to the <body> section, you'll notice four <p> elements. Two of these paragraphs have the class attribute set to "highlight". This means that we're applying the styles defined in the .highlight class to these specific paragraphs.
3. When the browser renders the page, it sees the class="highlight" attribute on the second and fourth paragraphs. It understands that these paragraphs should inherit the styles defined in the .highlight class.
4. As a result, the background color of these two paragraphs becomes blue, the text color turns white, and the text itself becomes bold. All of this is thanks to the CSS properties we defined earlier.

4.4.4 The title Attribute

The title attribute specifies a tooltip that is displayed when a user hovers over an element. You might recall we touched on this feature when discussing hyperlinks and the use of the title attribute to enhance user understanding and interaction.

For example:

index.html

```html
<button type="button" title="Click me for more information">
    Learn More</button>
```

Code/Preview

4.4.5 More Global Attributes to Explore

We've already looked at some key global attributes in HTML, but there's a lot more out there to discover. Remember, some attributes we haven't talked about yet might have popped up in earlier discussions about different HTML elements. If you're curious, experimenting with these attributes can be a great way to learn more about what you can do with HTML.

To spark your curiosity, here are a few more global attributes that can be really handy. Try playing around with them in your projects:

- accesskey – lets you set a keyboard shortcut to focus or activate an element,
- lang – sets the language for the content in an element,
- hidden – controls if an element is shown or hidden,
- dir – sets the direction of text in an element (like left-to-right or right-to-left),
- translate – tells browsers whether the content should be translated or not,
- onclick – runs a piece of JavaScript code when the element is clicked.

4.5 Event and Data Attributes

4.5.1 Understanding Event and Data Attributes

Event and data attributes are key to adding interactivity and personalized information to your web pages. Even without deep knowledge of JavaScript, it's useful to know what they can do.

Event attributes let HTML elements react to things users do, like clicking or hovering over them. Data attributes let you keep extra info inside HTML tags without changing how things look or act.

We'll give you a peek at these attributes and show some simple uses. To dive deeper into making them work with JavaScript, check out our Web Development Essentials: JavaScript course.

4.5.2 Event Attributes

Event attributes are used to define how HTML elements should respond to user interactions, such as mouse clicks or keyboard input. When an event is triggered, it executes a specified function or script.

Some of the most commonly used event attributes include:

- onclick – executes a script when the element is clicked,
- onmouseover – executes a script when the mouse pointer moves over the element,
- onmouseout – executes a script when the mouse pointer moves out of the element,
- onkeydown – executes a script when a keyboard key is pressed down,
- onkeyup – executes a script when a keyboard key is released.

For instance, using onclick, you can make a button show a message when clicked:

4.5.2 Event Attributes

index.html

```html
<button onclick="alert('Hello, World!')">Click Me</button>
```

Code/Preview

4.5.3 Data Attributes

Data attributes are used to store extra information about an HTML element that is not displayed to the user.

This information can be used for scripting or styling purposes (you can store information such as IDs, values, or other data that may be useful in web development), and can be accessed and manipulated via JavaScript or CSS.

You start these attributes with data- and add a name that makes sense for what you're storing, like data-recipe-id for a unique recipe number.

Imagine a cooking website where users can save recipes. You can use data attributes to keep each recipe's unique ID right in the HTML:

4.5.3 Data Attributes

index.html

```html
<ul>
    <li data-recipe-id="001">
        <h2>Spaghetti with Meatballs</h2>
        <p>A classic Italian dish that's perfect for a cozy night in.</p>
        <button class="save-recipe">Save Recipe</button>
    </li>
    <li data-recipe-id="002">
        <h2>Chicken Tikka Masala</h2>
        <p>A flavorful Indian curry that's sure to please.</p>
        <button class="save-recipe">Save Recipe</button>
    </li>
    <li data-recipe-id="003">
        <h2>Beef and Broccoli Stir Fry</h2>
        <p>A quick and easy weeknight dinner that's packed with flavor.</p>
        <button class="save-recipe">Save Recipe</button>
    </li>
</ul>
```

In this example, each recipe is represented by an li element with a data-recipe-id attribute that stores the unique ID of the recipe. When the user clicks the "Save Recipe" button, a JavaScript function could be called that retrieves the recipe ID from the data attribute and sends it to your backend API in order to add it to the user's recipe box.